

BEGINNER FRIENDLY

Tech Book Store Practice API

A Beginner's Step by Step Guide to API Test Automation with Playwright

Start from zero. No prior API knowledge needed.

Run a real API on your own computer, send your first request,
then automate your first tests with Playwright, one tiny step at a time.

Prepared by Manjunath N P

linkedin.com/in/manjunathnp

What is inside this guide

- Part 1.** The absolute basics, explained in plain words
- Part 2.** Set up your practice lab, step by tiny step
- Part 3.** Send your very first request by hand
- Part 4.** Explore safely with the built in documentation page
- Part 5.** Your first automated test with Playwright
- Part 6.** Running the tests in VS Code, click by click
- Part 7.** Running the Postman collection, click by click
- Part 8.** Read a real test, line by line
- Part 9.** The complete catalog of tests (positive and negative)
- Part 10.** Four advanced ideas, made simple
- Part 11.** A plain word glossary
- Part 12.** When something goes wrong

Part 1. The absolute basics, explained in plain words

If you have never tested an API before, this part gives you the whole picture in a few minutes. Read it once, slowly. You do not need to memorize anything. Every term here appears again later with an example.

What is an API?

API stands for Application Programming Interface. That sounds heavy, so here is the simple version: an API is a way for one program to ask another program to do something, using messages instead of buttons and screens.

In everyday terms

Think of a restaurant. You do not walk into the kitchen and cook. You give your order to a waiter, the kitchen prepares it, and the waiter brings back your food. An API is the waiter. Your program places an order (a request), the server does the work, and it sends back a result (a response).

What is a request and a response?

Every interaction with an API is one request and one response.

- A **request** is the message you send. It says what you want, for example "give me book number 1" or "create a new book with this title and price".
- A **response** is the message you get back. It contains a result and a short status number that says how it went.

What is a method?

Every request carries a method, which is a single word describing the kind of action. You will use five of them.

Method	Means	Example
GET	Read something, change nothing	Get the list of books
POST	Create something new	Add a new book
PUT	Replace something completely	Replace a whole book record
PATCH	Change part of something	Change only a book's price
DELETE	Remove something	Delete a book

What is a status code?

Every response comes with a three digit number that summarizes the outcome. You do not need all of them, just the families.

Range	Meaning	Common ones you will see
2xx	Success	200 ok, 201 created, 204 done with no content
4xx	You made a mistake	400 bad request, 401 not logged in, 403 not allowed, 404 not found, 409 conflict, 422 invalid fields, 429 too many requests
5xx	The server had a problem	500 internal error

Why this matters

Most of API testing is checking that you get the right status code for a given request. A good test does not only check the happy path (a 200). It also checks that a bad request gets the correct error, for example that creating a book without logging in returns 401, not a success.

What is JSON?

JSON is the format the messages are written in. It is just text, arranged as names and values inside curly braces. Here is a tiny book in JSON.

```
{
  "id": 1,
  "title": "Playwright End to End Testing",
  "price": 19.99,
  "inStock": false
}
```

You can read it left to right: the id is 1, the title is that text, the price is 19.99, and inStock is false. That is all JSON is. Curly braces hold an object. Square brackets hold a list. Text goes in double quotes. Numbers and true or false do not.

What is a header?

A header is a small label attached to a request or response, separate from the main body. Two headers matter most for you.

- **Content-Type** tells the server what kind of body you are sending. When you send JSON, you set it to `application/json`.

- **Authorization** carries your login token so the server knows who you are.

So what is API test automation?

It is writing small programs that send requests to an API and automatically check the responses, instead of you clicking through screens by hand. A test might log in, create a book, then confirm the book reads back correctly. If the API ever breaks, the test fails and tells you at once. That is the whole job, and this project is where you will practise it safely.

Part 2. Set up your practice lab, step by tiny step

Do these in order. Each step is small. After this part you will have a real API running on your own computer.

The terminal, in one paragraph

The terminal is a window where you type commands instead of clicking. On Windows, open the Start menu, type **PowerShell**, and press Enter. On a Mac, open Spotlight with Command and Space, type **Terminal**, and press Enter. A window opens with a blinking cursor. That is where every command below goes. You type the command, then press Enter.

1 Install Node.js

Node.js is the program that runs the API. Open a web browser, go to **nodejs.org**, and download the version labelled LTS (it means long term support, the stable one). Run the installer and keep clicking Next with the default choices. When it finishes, close and reopen your terminal.

Check it worked by typing this and pressing Enter:

```
node --version
```

You should see a version number such as `v20.11.0`. Any number that starts with 18, 20, or higher is fine. If you see a number, Node is installed.

2 Get the project onto your computer

Unzip the project folder somewhere easy to find, for example your Desktop. You will get a folder named `Tech-Book-Store-Upgrade`. It contains a clean `application` folder and a separate `automation` folder.

3 Move into the API folder

In the terminal, type `cd` (which means change directory), a space, then the path to the API inside the project. It will look something like this:

```
cd Desktop/Tech-Book-Store-Upgrade/application/api
```

4 Start the server

Type this and press Enter:

```
node server.js
```

The window prints a short banner and then seems to pause. That pause is good: it means the server is running and waiting for requests. You will see the addresses to use and the login accounts:

```
Base URL    : http://127.0.0.1:3000
API docs    : http://127.0.0.1:3000/docs

admin@bookstore.test / admin123      (role: admin)
user@bookstore.test / password123    (role: user)
```

Do not close this window

The server runs only while this terminal window stays open. Leave it running and open a second terminal window for the next commands. To stop the server later, click back into this window and press the Control key and C together.

The address 127.0.0.1 always means "this same computer". Port 3000 is just the door number the server listens on. So `http://127.0.0.1:3000` means "my own computer, door 3000".

Part 3. Send your very first request by hand

You have a running API. Let us talk to it. There are three easy ways, from simplest to most useful.

Way one: your web browser

A browser sends a GET request when you visit an address. Open a new browser tab and type this in the address bar:

```
http://127.0.0.1:3000/api/books?limit=3
```

Press Enter. You will see a block of JSON with three books inside a `data` list, and a `meta` block below it. That is a real API response. You just made your first request.

Way two: the built in documentation page

Open `http://127.0.0.1:3000/docs` in your browser. This is a friendly page that lists every endpoint and lets you send requests by clicking. Part 4 walks through it. For most beginners this is the nicest way to explore.

Way three: the curl command

curl is a small tool for sending requests from the terminal. It comes built in on modern Windows and Mac. In your second terminal window, try this:

```
curl http://127.0.0.1:3000/api/books/1
```

You get back book number 1 as JSON. To create a book you must prove you are an admin, which takes two steps. First, log in and copy the access token from the reply:

```
curl -X POST http://127.0.0.1:3000/api/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"email":"admin@bookstore.test","password":"admin123"}'
```

The reply contains a long `accessToken`. Copy it. Then paste it into the next command where it says `PASTE_TOKEN_HERE`:

```
curl -X POST http://127.0.0.1:3000/api/books \  
-H "Authorization: Bearer PASTE_TOKEN_HERE" \  
-H 'Content-Type: application/json' \  
-d '{"title":"My First Book","authorId":1,"categoryId":1,"price":19.99}'
```

You get back the new book with a fresh id and a status of 201, which means created. Congratulations: you have now read and created data through an API.

In everyday terms

Logging in and getting a token is like collecting a wristband at the entrance of an event. After that, you flash the wristband (the token in the Authorization header) instead of showing your ticket again at every stall.

Part 4. Explore safely with the built in documentation page

Open `http://127.0.0.1:3000/docs`. This page is organized like the professional API docs testers use every day. Here is how to drive it, with nothing left out.

1 Find your way around

On the left is a list of sections (Auth, Books, Cart, and so on) with a filter box at the top. In the middle is the same list as clickable rows. Each row is colored by its method: blue for GET, green for POST, orange for PUT, teal for PATCH, and red for DELETE.

2 Log in with one click

Click the green **Authorize** button at the top right. A small window opens. Press **Log in as admin**. It fills your token behind the scenes and shows that you are signed in. Press **Close**. You can now use the admin only actions.

3 Open an endpoint and read it

Click any row, for example the blue `GET /api/books`. It expands to show what it does, the parameters you can use, and the possible responses with their status codes. This is how you learn an endpoint before testing it.

4 Try it out for real

Inside the expanded endpoint, press **Try it out**. Fill any fields you like (or leave them blank), then press **Execute**. The real request goes to the server and the real response appears right there, including the status code and the JSON body. Nothing is pretend.

5 Reset whenever you want

If you create or delete things and want a clean slate, open Authorize again and press **Reset data**. The original 60 books come back. You can do this as often as you like.

A good first exploration

Try these in order on the docs page: run `GET /api/books` and read the list. Then Authorize as admin. Then run `POST /api/books` to create one. Then `GET /api/books/61` to read your new book. Then `DELETE /api/books/61` and try `GET /api/books/61` again to see the 410 Gone. Finally, Reset data. That single loop touches reading, creating, deleting, and error handling.

Part 5. Your first automated test with Playwright

Doing things by hand is great for learning. Automation is where testing becomes a real skill. Playwright is a free tool that sends requests and checks the responses for you, again and again, in seconds.

What is Playwright, simply?

Playwright is a testing tool made by Microsoft. People often use it to click around websites, but it also has a request client for testing APIs directly, with no browser involved. That is what this project uses, so it is fast and needs no browser download.

1 Install the test tools

Open a terminal in the separate `automation` folder. From the API folder used earlier, this command takes you there:

```
cd ../../automation
npm install
```

This reads the project's list of tools and installs Playwright and TypeScript into a local folder. You only do this once.

2 You do not start the server yourself

For the tests, the project is set up to start the server for you automatically and stop it when the tests finish. So you can close the server window from earlier if you like. The tests handle it.

3 Run the gentlest lesson first

Type this:

```
npm run test:smoke
```

Playwright quietly starts the server, sends a few simple requests, checks the replies, and prints a green list of passing tests. Seeing that green list is your first automated test run.

4 Run the full course

When you are ready, run everything:

```
npm test
```

Then open the nicely formatted report in your browser with:

```
npm run test:report
```

The lessons, in the order to learn them

The tests are grouped into folders that build on each other. Run them one at a time as you learn.

Command	What you practise
npm run test:smoke	Send a request, check a status and one field
npm run test:crud	Create, read, update, and delete a book
npm run test:query	Paging, sorting, filtering, and search
npm run test:auth	Tokens, roles, refresh, logout, API key, Basic auth
npm run test:chaining	Cart to order flows and safe retries
npm run test:negative	Every error on purpose
npm run test:advanced	Version conflicts, soft delete, rate limits

Part 6. Running the tests in VS Code, click by click

The terminal works, but many people prefer a friendly editor with buttons. VS Code (short for Visual Studio Code) is a free editor from Microsoft that shows your tests with little green play buttons. Here is the whole setup and run, with nothing skipped.

1 Install VS Code

Open a browser and go to **code.visualstudio.com**. Download the version for your system (Windows or Mac) and run the installer with the default choices. Open VS Code when it finishes.

2 Open the project folder

In VS Code, click the top menu **File**, then **Open Folder**. Find and select `Tech-Book-Store-Upgrade`, then click Open. The left panel shows the separate `application` and `automation` folders. If a bar asks whether you trust the authors, click **Yes, I trust the authors**.

3 Open a terminal inside VS Code

Click the top menu **Terminal**, then **New Terminal**. A terminal opens at the project root. Move into `automation` and run the one-time install:

```
cd automation
npm install
```

4 Install the Playwright extension (recommended)

Click the Extensions icon on the far left (it looks like four squares). In the search box type **Playwright**. Install the one named **Playwright Test for VSCode** made by Microsoft. This adds a Testing panel with run buttons.

5 Run tests with the green play buttons

Click the **Testing** icon on the far left (a flask or beaker shape). You will see the test files in a tree. Hover over any test or folder and a green triangle appears. Click it to run just that test or that whole group. Green ticks mean pass, red crosses mean fail. Click a test name to jump straight to its code.

6 Run everything, or run from the terminal

To run the full suite, press the top level play button in the Testing panel, or use the terminal at the bottom:

```
npm test
```

To run only one lesson, use its script, for example:

```
npm run test:auth
```

7 Debug a failing test, step by step

If a test fails and you want to watch it run slowly, click just to the left of a line number to place a red dot (a breakpoint). Then, in the Testing panel, right click the test and choose **Debug Test**. Execution pauses at your red dot so you can inspect what happened. This is the calm way to understand a failure.

Try the visual UI mode

For a guided, visual experience, run `npm run test:headed:ui` in the terminal. A window opens where you can pick tests, watch them run, and see each request and response in a timeline. It is the friendliest way to learn what each test does.

Part 7. Running the Postman collection, click by click

Postman is a popular visual tool for sending requests and running saved checks by clicking, with no coding at all. The project ships a ready made collection so you can test the whole API in Postman in minutes. Make sure the server is running first (Part 2, step 4).

1 Install Postman

Open a browser, go to **postman.com/downloads**, download Postman for your system, and install it. Open it when done. You can skip creating an account: look for a small **Skip and go to the app** or **Continue without an account** link.

2 Import the collection and the environment

In Postman, click the **Import** button (top left). Drag these two files from `application/api/postman` into it, then confirm:

- `TechBookStore.postman_collection.json` (the requests and their checks)
- `TechBookStore.postman_environment.json` (the base address)

3 Select the environment

At the top right there is a dropdown that usually says **No Environment**. Click it and choose **Tech Book Store (Local)**. This tells Postman that the base address is `http://127.0.0.1:3000`. Nothing works correctly until you do this.

4 Send your first request

On the left, expand the collection **Tech Book Store Practice API**, open the folder **01 Health and Info**, and click **Health**. Press the blue **Send** button. The response appears in the lower panel. Click the **Test Results** tab there to see the checks pass with green ticks.

5 Log in so later requests work

Open the **00 Setup** folder and send **Reset data** once. Then open **02 Auth** and send **Login as admin**, **Login as user**, and **Login as shopper (Sam)**. These quietly save your tokens into the collection so the Books, Cart, and Orders folders can use them. You do not copy anything by hand.

6 Run the entire collection at once

To run everything in order, hover over the collection name **Tech Book Store Practice API**, click the three dots, and choose **Run collection**. The Collection Runner opens. Keep the order as shown, make sure the environment is still selected, and press **Run Tech Book Store Practice API**. Postman fires all 68 requests and shows a full pass or fail summary.

7 Read the results

Each request shows how many checks passed. Green means pass, red means fail, with the reason next to it. A clean run is all green. If you see red, the most common cause is forgetting to select the environment (step 3) or not running the login requests first (step 5).

In everyday terms

The collection is like a recipe card box with the cards already in order. The first card preps the kitchen (reset), the next few sign you in, and each later card assumes that prep is done. Run them in order and dinner comes out right every time.

Part 8. Read a real test, line by line

Here is one real test from the smoke lesson. Do not worry about writing it yet. Just read the explanation next to each idea. Once this makes sense, every other test in the project will look familiar.

```
test('health endpoint responds ok', async ({ request }) => {  
  const res = await request.get('/health');  
  expect(res.status()).toBe(200);  
  
  const body = await res.json();  
  expect(body.status).toBe('ok');  
});
```

- `test('...', ...)` defines one test and gives it a name. The name is what you see in the pass or fail list.
- `async` and `await` simply mean "wait for this to finish before moving on". Network calls take a moment, so we wait for each one.
- `request.get('/health')` sends a GET request to the health endpoint. The result is stored in `res`, short for response.
- `expect(res.status()).toBe(200)` is a check. It reads "I expect the status to be 200". If it is not, the test fails.
- `res.json()` turns the response body into an object you can read.
- `expect(body.status).toBe('ok')` checks that the field named status equals the text ok.

That is the entire pattern of API testing: send a request, then write one or more expect checks about the response. Everything else is variations on this. A negative test looks the same, but the expected status is an error such as 401 or 404.

The one habit worth copying

The project logs in once and builds a helper that carries your token automatically, so tests do not repeat the login. You will see helpers named `adminApi` and `shopperApi` in the tests. When you write your own tests, copy that idea. It keeps tests short and clean.

Part 9. The complete catalog of tests

This is the full, exhaustive list of everything the project checks, so you know exactly what is covered. Each Playwright test is marked **POSITIVE** (it confirms a correct action succeeds) or **NEGATIVE** (it confirms a wrong action is refused with the right error). Testing both kinds is what separates a beginner from a professional.

Playwright tests: 50 in total (27 positive, 23 negative). Postman requests: 68 in total, each with its own checks. A built in checker (node tools/verify.js) runs 64 more assertions against the server.

Smoke (first checks)

Type	What the test confirms
POS	The health endpoint replies with status ok.
POS	The API index lists the available resources.
POS	The books list comes back in the standard data plus meta shape.

Books: create, read, update, delete

Type	What the test confirms
POS	Read a seeded book by its id and confirm it has an ETag.
POS	An admin creates a book and gets 201 with a Location header.
POS	A PATCH changes one field and the version number goes up by one.
POS	A PUT replaces the whole book record.
POS	An admin deletes a book, then reading it returns 410 Gone.

Books: list query features

Type	What the test confirms
POS	Page and limit values are honoured, and the seed has 60 books.
POS	A huge limit is capped at the server maximum of 100.
POS	Sorting by price, highest first, returns items in that order.
POS	Filtering by in stock true returns only in stock books.
POS	Filtering by a price range returns only books inside the range.
POS	Search finds books ignoring upper or lower case.
POS	Asking for only id and title returns just those two fields.

Authentication and roles

Type	What the test confirms
POS	Login returns both an access token and a refresh token.
NEG	A protected route with no token is rejected with 401.
POS	The me endpoint returns the logged in user.
NEG	A normal user creating a book is refused with 403.
NEG	Refresh returns new tokens, and the old refresh token stops working.
NEG	Logout makes the access token stop working (401 afterwards).
POS	Register creates a normal user and returns tokens.
NEG	Registering an email that already exists is refused with 409.

Other login styles

Type	What the test confirms
NEG	An API key request with no key is rejected with 401.
POS	An API key request with the correct key is accepted.
NEG	An API key request with a wrong key is rejected with 401.
POS	Basic auth with correct credentials is accepted.
NEG	Basic auth with wrong credentials is rejected with 401.

Chaining: cart to order

Type	What the test confirms
POS	Add items, check out, read the order back, then cancel it.
POS	Sending the same idempotency key twice returns the same order.
NEG	Checking out with an empty cart is refused with 409.
NEG	A different user cannot read your order (403).

Chaining: books and reviews

Type	What the test confirms
POS	Create a book, post a review to it, then read the review back.
NEG	Reviewing the same book twice as one user is refused with 409.
NEG	Reviewing without logging in is refused with 401.

Error and boundary handling

Type	What the test confirms
NEG	Reading a book id that does not exist returns 404.
NEG	Calling a route that does not exist returns 404.
NEG	Using the wrong method returns 405 with an Allow header.
NEG	Sending a non JSON body returns 415.
NEG	Sending broken JSON returns 400.
NEG	Invalid fields return 422 with a details list of what is wrong.
NEG	A book pointing at a missing author returns 422 naming authorId.
NEG	Creating a book with no token returns 401.
POS	The status generator returns whatever code you ask for.

Advanced behaviours

Type	What the test confirms
POS	Reading gives an ETag, and an update with a matching ETag succeeds.
NEG	An update with a stale ETag is refused with 412.
NEG	Going over the rate limit returns 429 with a Retry-After header.
POS	The delay endpoint really waits before replying.
NEG	A deleted book returns 410 and is hidden from the normal list.
POS	Restore brings a soft deleted book back so it reads with 200 again.

The Postman collection, folder by folder

The Postman collection covers the same ground with clickable requests, each carrying its own automatic checks. Run the folders top to bottom. The counts show how many requests are in each folder.

Folder	Requests	What it covers
00 Setup	1	Reset the data to a known starting point.
01 Health and Info	2	Confirm the server is alive and lists its resources.
02 Auth	10	Register, log in as admin, user, and shopper, read the profile, refresh tokens, and try the wrong password, API key, and Basic auth.
03 Books	18	List, page, sort, search, filter, sparse fields, get one, and the full admin lifecycle including 401, 403, 422, 412, soft delete, restore, 405.
04 Authors, Categories, Publishers	6	List and read the simple resources, create one as admin, and confirm a normal user is refused with 403.
05 Reviews	4	Create a review under a book, list it, get a 409 on a duplicate, and update your own review.
06 Cart and Wishlist	7	Empty the cart, add items, view totals, change quantity, and add to the wishlist.
07 Orders	8	Check out with an idempotency key, replay it, read and list orders, get a 403 as another user, cancel, and see 409 on a second cancel or empty cart.
08 Negative and Status Codes	5	Trigger 415, 400, and generate 418, 503, and 204 on demand.
09 Advanced	7	Exercise the delay endpoint and push the rate limit until it returns 429.

Every Postman request, by name

For completeness, here is every single request in the collection. A name ending in a status code (for example 401, 403, 409) is a negative test that deliberately triggers that error. All the others confirm a correct action. You can run any of these on its own, or run a whole folder, or run the entire collection.

00 Setup

- Reset data

01 Health and Info

- Health
- API index

02 Auth

- Register new user
- Login as admin
- Login as user
- Login as shopper (Sam)
- Get current user (me)
- Login with wrong password (401)
- Refresh tokens (rotation)
- API key endpoint (200)
- API key endpoint wrong key (401)
- Basic auth endpoint (200)

03 Books

- List books
- List books page 2 limit 5
- Sort by price desc
- Search playwright
- Filter inStock and price range
- Sparse fields id,title
- Get book 1 (capture ETag)
- Get missing book (404)

- Create book without token (401)
- Create book as user (403)
- Create book invalid (422)
- Create book (admin), capture id and ETag
- Update with stale If-Match (412)
- Update with correct If-Match (200)
- Soft delete book (204)
- Get soft-deleted book (410)
- Restore book (200)
- Method not allowed on collection (405)

04 Authors, Categories, Publishers

- List authors
- Get author 1
- Create author (admin)
- List categories
- List publishers
- Create category as user (403)

05 Reviews

- Create review under book 45
- List reviews for book 45
- Duplicate review (409)
- Update own review

06 Cart and Wishlist

- Empty cart first
- Add book 2 x2 to cart
- Add book 3 to cart
- View cart
- Update cart item quantity
- Add to wishlist
- View wishlist

07 Orders

- Checkout (create order) with Idempotency-Key
- Replay checkout (same key, same order)
- Get order
- List my orders
- Another user cannot view my order (403)
- Cancel order
- Cancel again (409)
- Checkout empty cart (409)

08 Negative and Status Codes

- 415 wrong content type
- 400 bad JSON body
- Status generator 418
- Status generator 503
- Status generator 204

09 Advanced

- Delay 800ms
- Rate limit hit 1
- Rate limit hit 2
- Rate limit hit 3
- Rate limit hit 4 (likely 429)
- Rate limit hit 5 (likely 429)
- Rate limit hit 6 (likely 429)

Coverage at a glance

Between the 50 Playwright tests, the 68 Postman requests, and the 64 built in server assertions, every endpoint, every status code from 200 to 429, both login styles, all the chaining flows, and all the advanced behaviours are exercised in both their correct and their failing forms. That is well over one hundred concrete, automated scenarios.

Part 10. Four advanced ideas, made simple

Version conflicts, and the ETag

Imagine two people edit the same book at the same time. Both read version 1. The first saves a change. The second, still holding version 1, saves and quietly erases the first person's work. To prevent this, every book carries a small version tag called an ETag. When you save, you may send the tag you last saw. If the book changed since then, your tag is out of date and the server refuses with status 412, so you re read and try again.

In everyday terms

It is like editing a shared paper list with a version number in the corner. Before you cross anything off, you check the number matches the copy you took. If it does not, someone edited it while you were away, so you fetch the fresh copy first instead of scribbling over their work.

Safe retries, and the idempotency key

You press check out, the network stalls, and you are unsure if the order went through. If you press again, will you be charged twice? An idempotency key prevents that. It is a unique string you attach to the checkout. If the server sees the same key twice, it returns the order it already made instead of making a second one.

In everyday terms

It is like the call button for a lift. Pressing it five times does not summon five lifts. The first press counts, and the rest do nothing new.

Delete without destroying, the soft delete

Deleting a book here does not erase it. It marks the book as deleted and hides it from normal lists. Reading it afterwards gives status 410, which means it existed and is now gone, a clearer message than a plain 404. Because the record still exists, a restore action brings it right back.

In everyday terms

This is the recycle bin on your computer. Deleted files are not shredded on the spot; they move out of sight and can be restored if you change your mind.

Being asked to slow down, rate limiting

One endpoint accepts only a few requests in a short window. Go over, and it replies with status 429, meaning too many requests, plus a Retry-After header telling you how many seconds to wait.

Handling this politely, by pausing and trying again, is a skill every API client needs.

In everyday terms

Think of a ride with a gate that lets a set number of people through each minute. Arrive too fast and the attendant asks you to wait a moment. Retry-After is the attendant telling you exactly how long.

Part 11. A plain word glossary

API

A way for programs to talk to each other using messages. Your practice target is an API.

Endpoint

One specific address and method you can call, for example GET /api/books. An API is a collection of endpoints.

Request

The message you send to an endpoint, describing what you want.

Response

The message you get back, containing a status code and usually a JSON body.

Method

The action word on a request: GET, POST, PUT, PATCH, or DELETE.

Status code

A three digit number that summarizes the outcome. 2xx is success, 4xx is your mistake, 5xx is the server's problem.

JSON

The text format used for message bodies, made of names and values inside curly braces.

Header

A small label on a request or response, such as Content-Type or Authorization.

Body

The main content of a request or response, usually JSON.

Token

A long string you receive after logging in. You attach it to later requests to prove who you are.

Bearer token

The style of token used here. It travels in the header as "Authorization: Bearer your-token".

Access token

A short lived token used on normal requests. It expires after fifteen minutes by default.

Refresh token

A longer lived token whose only job is to get you a new access token without logging in again.

Authentication

Proving who you are. A missing or bad token gives a 401.

Authorization

Checking what you are allowed to do. A valid user doing a forbidden action gets a 403.

Positive test

A test that confirms a correct action succeeds.

Negative test

A test that confirms a wrong action fails with the right error.

Chaining

Using the result of one request as the input to the next, for example create a book then review it.

ETag

A version tag on a record used to prevent two people overwriting each other's changes.

Idempotency key

A unique string that lets you retry an action safely without doing it twice.

Soft delete

Marking a record as deleted and hiding it, rather than erasing it, so it can be restored.

Rate limit

A cap on how many requests you may send in a time window. Going over returns 429.

Playwright

The tool used here to send requests and check responses automatically.

Postman

A visual tool for sending requests and running collections of checks by clicking.

Seed data

The fixed starting data the server loads every time it starts or is reset.

Part 12. When something goes wrong

What you see	What it means and what to do
node is not recognized	Node.js is not installed, or the terminal was open before you installed it. Install from nodejs.org , then close and reopen the terminal.
Cannot connect, or the page will not load	The server is not running. In <code>application/api</code> , run <code>npm start</code> again and leave that window open.
Every write returns 401	You have no token, or it expired after fifteen minutes. Log in again, or on the docs page press Authorize and log in as admin.
A write returns 403	You are logged in as a normal user. Creating or changing catalog items needs the admin account.
A request with a body returns 415	You forgot the header that says the body is JSON. Add Content-Type application/json.
An update returns 412	Your ETag is out of date. Read the item again to get the current ETag, then retry the update.
The limited endpoint keeps returning 429	That is expected once you pass the limit. Wait the seconds shown in Retry-After, or reset the data.
Tests behave differently each run	Reset the data at the start so every run begins from the same 60 books. The tests already do this where needed.
npm install seems stuck	The first install downloads a few files and can take a minute on a slow connection. Give it time. If it fails, check your internet and run it again.

A closing thought

The fastest way to get good at API testing is steady repetition against a target that never surprises you for the wrong reasons. Start the server, pick one scenario from Part 9, reproduce it by hand on the docs page, then find or write the automated test for it and watch it go green. Then pick the next one. Small wins add up quickly, and before long the whole catalog will feel easy.